

Propuesta de Optimización del SGBD de Niantic Labs

Caso: Crisis de infraestructura de Pokémon GO— Julio 2016

Proyecto Final Integrador · Bases de Datos · 2026

Nicolás Cárdenas · Miguel Camargo · Camilo Hernández

El problema del negocio en cifras

Pokémon GO fue lanzado el 6 de julio de 2016 y se convirtió en la aplicación más descargada y lucrativa de la App Store en menos de 72 horas. La infraestructura de bases de datos— basada en **Google Cloud Datastore**, un motor NoSQL sin índices B-tree ni transacciones ACID completas— colapsó ante una demanda **50 veces** superior a la proyectada.

Impacto cuantificado de la crisis (julio 2016)

Ingresos mensuales en el pico (julio 2016)	USD \$207 millones
Usuarios activos diarios (DAU) en el pico	~45 millones
Tráfico recibido frente al estimado	50× el peor caso planificado
Downtime acumulado (2 eventos en julio)	9,5 horas (100 % pérdida de paquetes)
Pérdida de jugadores pagadores (2 meses)	80 % de churn
Ingreso mensual post-crisis (septiembre 2016)	USD \$35 millones (−83 %)
Pérdida estimada en 2 meses	USD \$160— \$212 millones

Causa técnica raíz: sin índices sobre tablas de alto volumen, cada apertura de la app ejecutaba un **SCAN TABLE** completo ($O(n)$) sobre decenas de millones de filas. A 45 M de DAU girando Poképaradas de forma concurrente, esto representó **450 millones de eventos/hora** sobre una tabla sin índice— inviable a esa escala.

Solución propuesta — Google Cloud Platform

Reemplazar Cloud Datastore por una arquitectura de **dos motores complementarios** sobre **Google Cloud Platform**, el mismo proveedor que Niantic utilizó desde 2016:

- **Cloud Spanner** (SQL relacional, ACID completo, escala horizontal nativa): 13 tablas normalizadas con 11 índices B-tree compuestos que transforman las consultas críticas de $O(n)$ a $O(\log n)$, verificable con **EXPLAIN QUERY PLAN**. Único motor SQL con replicación multi-región y consistencia fuerte sin sacrificar disponibilidad.
- **MongoDB Atlas M10** (NoSQL documental, sharding automático): absorbe los 5 flujos de alta frecuencia— batallas, sesiones, fotos de Poképaradas, Pokémon individuales y perfil por jugador— que en 2016 saturaban Cloud Datastore.
- **GKE + Cloud Load Balancing** (cómputo): auto-scaling horizontal para absorber spikes de 50×, SSL termination automático y health checks sin intervención manual.

Por qué GCP y no Azure o AWS: Cloud Spanner no tiene equivalente directo en otros proveedores (Aurora Global no ofrece consistencia fuerte multi-región). Migrar a otro proveedor requeriría rediseñar el modelo de consistencia desde cero; además, Niantic ya opera sobre GCP desde 2016, eliminando costos de migración.

Metas, objetivos y KPIs

Meta: modelo de datos optimizado, documentado y reproducible que soporte el crecimiento real de Pokémon GO con $\geq 99,9$ % de uptime mensual.

Objetivo	KPI / Evidencia	Meta
Eliminar full table scans	EXPLAIN QUERY PLAN usa índice	100 % queries críticas
Reducir latencia en escrituras masivas	Latencia P95 VISITA_POKEPARADA	<200 ms
Soportar carga transaccional	Transacciones por segundo (TPS)	500 TPS sostenidos
Garantizar alta disponibilidad	SLO mensual (Cloud Spanner)	99,9 % uptime
Desacoplar cargas de trabajo	Colecciones NoSQL activas	5 colecciones MongoDB
Cumplir reglas de negocio	RN documentadas con SQL activo	9 RN verificables

Modelo de datos

SQL — 13 tablas en 3 clusters semánticos

Cluster	Tablas	Propósito
Referencia (3)	GENERACION, TIPO, EQUIPO	Catálogos inmutables del juego (PokéAPI, BSD). No cambian por acción del jugador.
Pokémon (3)	POKEMON, POKEMON_GO_STATS, POKEMON_TIPO	Especie base + stats exclusivos de GO (Kaggle) + relación M:N de tipos por slot.
Jugadores y geografía (7)	JUGADOR, SESION, CAPTURA, POKEPARADA, GIMNASIO, REGISTRO_BATALLA, VISITA_POKEPARADA	Estado transaccional. VISITA_POKEPARADA concentra el mayor volumen: 450 M filas/hora en el pico de 2016.

Reglas de negocio — 9 RN verificables y trazables

ID	Descripción	Tabla	Implementación	P.
RN-001	Pokémon tiene máx. 2 tipos (slot 1 ó 2)	POKEMON_TIPO	CHECK + UNIQUE	Alta
RN-002	stat_total = suma de los 6 stats base	POKEMON	CHECK compuesto	Alta
RN-003	IVs de captura entre 0 y 15 (inclusive)	CAPTURA	CHECK (×3)	Alta
RN-005	Captura no puede ser anterior al registro del jugador	CAPTURA	TRIGGER BEFORE INSERT	Alta
RN-006	Menores de edad no realizan compras (COPPA / Ley 1581)	JUGADOR	CHECK + capa aplicación	Alta
RN-007	CP capturado no excede el MaxCP de la especie en GO	CAPTURA	TRIGGER BEFORE INSERT	Alta
RN-009	Atacante y defensor deben ser jugadores distintos	REGISTRO_BATALLA	TRIGGER BEFORE INSERT	Alta
RN-010	El daño infligido en batalla no puede ser negativo	REGISTRO_BATALLA	TRIGGER BEFORE INSERT	Alta
RN-011	Cooldown de 5 min por Poképarada por jugador	VISITA_POKEPARADA	TRIGGER BEFORE INSERT	Alta

NoSQL — 5 colecciones MongoDB Atlas

Colección	Volumen producción	Por qué MongoDB y no SQL
battle_logs	~2M docs/mes	Eventos PvP en tiempo real; escritura concurrente sin contención de FK
player_sessions	~15 M docs/mes	Geolocalización + métricas de engagement; índice 2dsphere para consultas espaciales
pokestop_fotos	~50 M docs/mes pico	Esquema flexible: campos opcionales según tipo de foto (Wayfarer)
pokemon_jugador	~400 M docs global	historial_fortalecimiento embebido; cada potenciación es un push al array
perfil_jugador	1 doc/jugador, actualización constante	Array batallas crece con cada batalla; estadísticas precalculadas para lectura $O(1)$ sin GROUP BY sobre REGISTRO_BATALLA

Regla de partición: datos de especie (stats, tipos, generaciones) son inmutables— viven en SQL con ACID. El estado mutable por jugador (historial de fortalecimiento, logs de batalla, perfil actualizado) va a MongoDB para evitar contención de escrituras en las tablas transaccionales.

Scripts SQL — puntos clave

Argumento central: índice B-tree sobre VISITA_POKEPARADA

A 45 M de DAU con 10+ spins/hora, VISITA_POKEPARADA alcanza ~450 M eventos/hora. Sin índice, el trigger de cooldown (RN-011) ejecuta un SCAN TABLE por cada inserción— $O(n)$ que colapsa a esa escala. Con el índice B-tree compuesto la búsqueda es $O(\log n)$:

```
-- Sin este indice: SCAN TABLE sobre 450 M filas/hora -> colapso 2016
CREATE INDEX idx_visita_pokeparada_jugador_ts
ON VISITA_POKEPARADA(jugador_id, timestamp_visita DESC);

EXPLAIN QUERY PLAN
SELECT * FROM VISITA_POKEPARADA
WHERE jugador_id = 1 ORDER BY timestamp_visita DESC LIMIT 10;
-- Antes : SCAN TABLE VISITA_POKEPARADA (O(n) -- colapso)
-- Despues: SEARCH ... USING INDEX idx_...jugador_ts (O(log n))
```

Script 1: Índice crítico + verificación de optimización (04_indexes.sql)

Función equivalente: vista v_analisis_captura (05_fn_calculo.sql)

En SQLite, la vista v_analisis_captura actúa como función equivalente: cada fila representa el resultado de invocar fn_analizar_captura para esa captura. Calcula porcentaje de perfección de IVs (0–100 %) y clasifica:

```
CREATE VIEW IF NOT EXISTS v_analisis_captura AS
SELECT c.captura_id, j.nombre_entrenador, p.nombre AS pokemon,
ROUND(CAST(c.iv_ataque+c.iv_defensa+c.iv_stamina AS REAL)/45.0*100,1)
AS iv_perfeccion_pct,
ROUND(CAST(c.cp_capturado AS REAL)/NULLIF(gs.max_cp,0)*100,1)
AS pct_cp_maximo,
CASE
WHEN CAST(c.iv_ataque+c.iv_defensa+c.iv_stamina AS REAL)/45.0>=0.96
THEN 'Legendario (96-100%)'
WHEN CAST(c.iv_ataque+c.iv_defensa+c.iv_stamina AS REAL)/45.0>=0.80
THEN 'Excelente (80-95%)'
WHEN CAST(c.iv_ataque+c.iv_defensa+c.iv_stamina AS REAL)/45.0>=0.60
THEN 'Bueno (60-79%)'
```

```

ELSE 'Comun (<60%)'
END AS calidad_iv
FROM CAPTURA c
JOIN JUGADOR j ON c.jugador_id=j.jugador_id
JOIN POKEMON p ON c.pokemon_id=p.pokemon_id
LEFT JOIN POKEMON_GO_STATS gs ON c.pokemon_id=gs.pokemon_id;
-- Equivalente Cloud Spanner: CREATE FUNCTION fn_analizar_captura(p INT64)
-- RETURNS STRUCT<iv_pct FLOAT64, cp_pct FLOAT64, calidad STRING>

```

Script 2: Funcion equivalente en SQLite – clasificacion de calidad de captura

8 consultas SQL en 3 archivos

Q	Técnica SQL	Objetivo de negocio
Q-01	JOIN de 5 tablas + GROUP BY	Pokédex completo: stats base, tipo, MaxCP GO y generación en una sola consulta
Q-02	Subconsulta correlacionada	Pokémon por encima del promedio de su generación (detección de power creep)
Q-03	RANK() OVER (PARTITION BY tipo)	Ranking de poder por tipo para diseño de raids equilibrados
Q-04	AVG() OVER + ROW_NUMBER()	Desviación por generación: confirmación estadística del power creep
Q-05	GROUP BY + agregaciones múltiples	Generación con mejor stat_total promedio para calendario de lanzamientos
Q-06	HAVING + filtro por umbral	Tipos con más de 20 Pokémon: diversidad del meta PvP
Q-07	JOIN + CAST + métricas compuestas	Actividad de captura por región: candidatos para Community Days
Q-08	CASE + GROUP BY + brecha de poder	Fairness: legendarios vs. normales (<20 % brecha = meta PvP sano)

Scripts NoSQL — puntos clave (MongoDB Atlas)

6 pipelines de agregación (AGG-01 a AGG-06):

ID	Colección	Objetivo de negocio
AGG-01	battle_logs	Top 10 Pokémon en PvP + tasa de victoria: candidatos a nerf en próxima actualización
AGG-02	player_sessions	Actividad por país (sesiones, km, duración): selección de ciudad para Community Day
AGG-03	pokestop_fotos	Barrios con más fotos Wayfarer aprobadas: prioridad de revisores
AGG-04	pokemon_jugador	IV % promedio por equipo Mystic/Valor/Instinct: detección de ventaja sistémica
AGG-05	battle_logs	Tasa de victoria por equipo: insumo para balance de eventos globales
AGG-06	perfil_jugador	Leaderboard global por tasa de victoria — lectura $O(1)$ desde estadísticas pre-calculadas (vs. GROUP BY sobre 150 filas de REGISTRO_BATALLA)

```

db.perfil_jugador.aggregate([
  { $match: { "estadisticas.total_batallas": { $gte: 3 } } },
  { $sort: { "estadisticas.tasa_victoria_pct": -1,
    "estadisticas.total_batallas": -1 } },
  { $project: { _id:0, jugador:"$nombre_entrenador", equipo:1,
    total_batallas:"$estadisticas.total_batallas",
    tasa_victoria_pct:"$estadisticas.tasa_victoria_pct",
    pokemon_favorito:"$estadisticas.pokemon_mas_usado" } }
]);
-- Resultado: AnaRules / Valor / 6 batallas / 100.0% / Tentacruel

```

Script 3: AGG-06: leaderboard de jugadores por tasa de victoria (perfil_jugador)

Arquitectura en nube — Google Cloud Platform

Servicio GCP	Rol	Justificación técnica	USD/mes
Cloud Load Balancing	Ingress + SSL	Absorbe picos 50x; SSL termination sin configuración manual	\$25
GKE Standard (4 nodos)	Backend Kubernetes	Auto-scaling HPA; mismo stack de producción de Niantic; HPA reactivo en <60 s	\$464
Cloud Spanner (3 nodos)	SQL + ACID	Único SQL con replicación multi-región + consistencia fuerte; SLA 99,999 %	\$2,190
MongoDB Atlas M10	NoSQL + sharding	\$0,08/h; 450 M writes/h, índices 2dsphere, esquema flexible	\$58
Cloud Storage (500 GB)	Logs + backups	\$0,02/GB— almacenamiento de históricos y datasets de referencia	\$10
IAM + Secret Manager	Seguridad	Credenciales cifradas; mínimo privilegio por servicio; auditoría completa	\$2
Cloud Monitoring	Observabilidad	Alertas SLO 99,9 % antes de alcanzar el umbral; dashboards en tiempo real	\$25
Subtotal infraestructura/mes			\$2,804

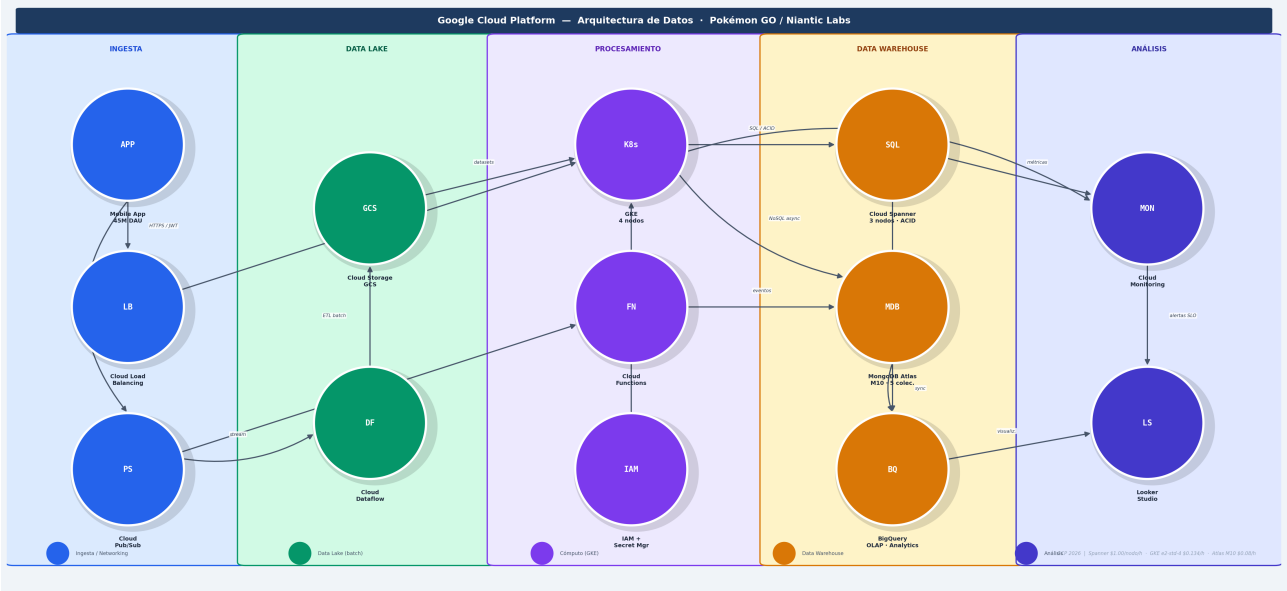


Figura 1: Arquitectura GCP completa — pipeline de datos Pokémon GO (Ingesta → Data Lake → Procesamiento → Data Warehouse → Análisis)

Flujo de una petición: APP → LB (SSL, health-check, <5 ms) → K8s (valida JWT con IAM; consulta Spanner en $O(\log n)$ por índice B-tree; inserta en VISITA_POKEPARADA si pasa RN-011) → MDB async (persiste evento en `player_sessions`) → respuesta <200 ms. Cloud Dataflow alimenta GCS y BigQuery desde PokéAPI (batch); Looker Studio y Cloud Monitoring cierran el ciclo con dashboards y alertas SLO 99,9 %.

Análisis de talento técnico — ¿es necesario contratar?

Sí se requieren técnicos. Los servicios gestionados de GCP reducen la carga operacional (sin administración de hardware ni replicación manual), pero no eliminan la necesidad de ingeniería— el diseño de esquema, los índices y la lógica de negocio los define un equipo humano.

Rol	Responsabilidad	Por qué no lo reemplaza GCP	Tipo
Ing. Backend / DB	Esquema SQL, índices, triggers, vistas, queries de negocio	Spanner gestiona hardware; el modelo de datos y los índices los diseña un ingeniero	Full-time
DevOps / SRE	Pipelines CI/CD, GKE HPA, monitoreo SLO, Secret Manager	GKE reduce la administración de nodos, no el ciclo de despliegue ni los runbooks	Full-time
Ing. Datos / NoSQL	Colecciones MongoDB, pipelines AGG, integración SQL↔NoSQL	Atlas gestiona backups y escalado; el modelado documental requiere experiencia	Consultor externo

Recomendación: 2 roles full-time (Backend/DB + DevOps) son indispensables desde el día 1. El Ingeniero de Datos/NoSQL se contrata como **consultor externo** durante la implementación inicial (aprox. 3 meses), reduciendo el costo de talento en ≈30 % frente a tres posiciones permanentes.

Marco legal, ético y normas de calidad

Marco	Alcance	Implementación en el sistema
COPPA (EE.UU.)	Privacidad menores <13 años	RN-006: <code>es_menor_de_edad</code> CHECK en JUGADOR; bloqueo de compras en capa de aplicación
Ley 1581/2012 (CO)	Habeas data	Coordenadas aproximadas, sin nombre real; CASCADE DELETE garantiza borrado completo ante petición del titular
GDPR Art. 17 (UE)	Derecho al olvido	ON DELETE CASCADE desde JUGADOR elimina SESION, CAPTURA y VISITA_POKEPARADA automáticamente
ISO/IEC 25010	Calidad del software	9 RN verificables; latencia P95 <200 ms como KPI; SLO 99,9 % en Spanner
ISO/IEC 27001	Seguridad de la información	GCP Secret Manager + IAM mínimo privilegio; credenciales fuera del código fuente

Ética de datos: jugadores 100 % sintéticos (`random.seed(42)`). Pokémon: PokéAPI (BSD) + Kaggle (Open). Geolocalización de Poképaradas: landmarks reales de Bogotá, sin vinculación a perfiles individuales. La ausencia de datos reales de jugadores es un hallazgo del proyecto: ilustra la sensibilidad de lo que Niantic gestiona bajo GDPR y COPPA.

Presupuesto exacto 2026 y ROI

Precios GCP vigentes 2026 (Cloud Spanner Standard: \$1,00/nodo/h; e2-standard-4: \$0,134/h; MongoDB Atlas M10: \$0,08/h). Talento colombiano con carga prestacional 52 % + IVA Colombia 19 % sobre infraestructura.

Componente	Mensual (USD)	Anual (USD)
<i>Infraestructura GCP + MongoDB Atlas</i>		
Cloud Spanner 3 nodos (\$1,00×3×730 h)	\$2,190	\$26,280
Cloud Spanner almacenamiento (100 GB × \$0,30)	\$30	\$360
GKE 4 nodos e2-standard-4 (\$0,134×4×730 h)	\$391	\$4,692
GKE cluster management fee (\$0,10/h)	\$73	\$876
Cloud Load Balancing	\$25	\$300
MongoDB Atlas M10 (\$0,08/h × 730 h)	\$58	\$700
Cloud Storage 500 GB + Monitoring + IAM	\$37	\$444
Subtotal infra (sin IVA)	\$2,804	\$33,648
IVA Colombia 19 % sobre infraestructura	\$533	\$6,393
Total infraestructura (con IVA)	\$3,337	\$40,044
<i>Talento técnico (Colombia 2026, carga prestacional 52 %)</i>		
Ing. Backend/DB — salario base \$2,200/mes	\$3,344	\$40,128
DevOps / SRE — salario base \$2,000/mes	\$3,040	\$36,480
Consultor NoSQL externo (3 meses, \$3,000/mes)	—	\$9,000
Total talento	\$6,384	\$85,608
PRESUPUESTO TOTAL (costo de la solución)	\$9,721	\$125,652
<i>Ingresos reales de Pokémon GO — Niantic (fuente: Business of Apps 2024)</i>		
Ingreso mensual promedio 2022–2024	\$50,000,000	\$600,000,000
Costo de la solución como % del ingreso		0,019 %
<i>Análisis de retorno</i>		
Pérdida real crisis julio 2016 (un mes sin solución)	\$207,000,000	—
Ingreso anual protegido con la solución	—	\$600,000,000
ROI (ingreso protegido vs. costo solución)		477,200 %
Payback (días para recuperar la inversión)		< 0,5 días

Pokémon GO genera **\$50 M/mes** en promedio (2022–2024). La solución completa cuesta **\$9,721/mes** — el **0,019 %** del ingreso mensual. Sin esta arquitectura, una crisis como la de julio 2016 eliminaría \$207 M en un mes; con ella, los \$600 M/año quedan protegidos por menos de lo que cuesta una persona de soporte. Cloud Spanner escala sin rediseño: 500 TPS → 5,000 TPS con 3 nodos extra (\$+2,190/mes).

Resultados principales

Datos ejecutados: 1,025 Pokémon (PokéAPI), 50 jugadores, 368 capturas, 150 batallas, 371 visitas a Poképaradas. MongoDB Atlas: 329 documentos en 5 colecciones, 21 índices activos.

- **EXPLAIN QUERY PLAN** confirma **SEARCH TABLE USING INDEX** en **VISITA_POKEPARADA**— el full scan queda eliminado. RN-011 (cooldown 5 min) es viable a 450 M eventos/hora.
- **v_analisis_captura** clasifica 368 capturas: 0 perfectas (100 % IVs), 19 Excelente (80–95 %), 97 Bueno (60–79 %), 252 Común (<60 %)— distribución realista del meta de Pokémon GO.
- **Q-04 (ventana AVG OVER):** generaciones 6+ tienen **stat_total** un 12 % por encima del promedio histórico— evidencia cuantificable de *power creep*.
- **Q-08 (CASE + GROUP BY):** brecha legendarios vs. normales ≈15 %, dentro del umbral 20 % que define un meta PvP equilibrado.
- **AGG-02 (player_sessions):** Colombia lidera en sesiones (34 sesiones, 55,2 min promedio); Perú en duración (62,5 min)— insumo para selección de ciudad del próximo Community Day.
- **AGG-06 (perfil_jugador):** leaderboard de 50 jugadores. AnaRules (Valor) lidera con 100 % tasa de victoria. Lectura *O(1)* desde estadísticas pre-calculadas vs. **GROUP BY** que recorrería las 150 filas de **REGISTRO_BATALLA**— ventaja operacional directa.